Labsheet 6 : BTCS301P-AS06

Computer Vision Lab

Khushi Choudhary CU240251564

In [ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
import urllib.request
import time

# Step 1: Download Sample Image from a Public URL using a browser User-Agent
image_url = "https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcQBVnv6ZzForb22sMpFGq3TmVrBYwxfgLJb3_bDrodSKQ&s=10"
req = urllib.request.Request(
    image_url,
    headers={'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)'}
)
with urllib.request.urlopen(req) as response:
    arr = np.asarray(bytearray(response.read()), dtype=np.uint8)
    img_bgr = cv2.imdecode(arr, -1)
# Resize for consistent processing speed across machines
img_bgr = cv2.resize(img_bgr, (500, 350))
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Step 2: Grayscale Conversion & Noise Reduction
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# Step 3: Global Thresholding
t_start = time.time()
_, global_thresh = cv2.threshold(blurred, 127, 255, cv2.THRESH_BINARY)
time_global = (time.time() - t_start) * 1000

# Step 4: Otsu's Thresholding
t_start = time.time()
otsu_val, otsu_thresh = cv2.threshold(blurred, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
time_otsu = (time.time() - t_start) * 1000

# Step 5: Adaptive Thresholding
t_start = time.time()
adaptive_thresh = cv2.adaptiveThreshold(
    blurred, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2
)
time_adaptive = (time.time() - t_start) * 1000

# Step 6: Watershed Segmentation
t_start = time.time()
_, thresh_inv = cv2.threshold(blurred, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(thresh_inv, cv2.MORPH_OPEN, kernel, iterations=2)
sure_bg = cv2.dilate(opening, kernel, iterations=3)
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist_transform, 0.4 * dist_transform.max(), 255, 0)
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)
_, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0
watershed_img = img_rgb.copy()
markers = cv2.watershed(watershed_img, markers)
watershed_img[markers == -1] = [255, 0, 0]  # Mark segmented boundaries red
time_watershed = (time.time() - t_start) * 1000

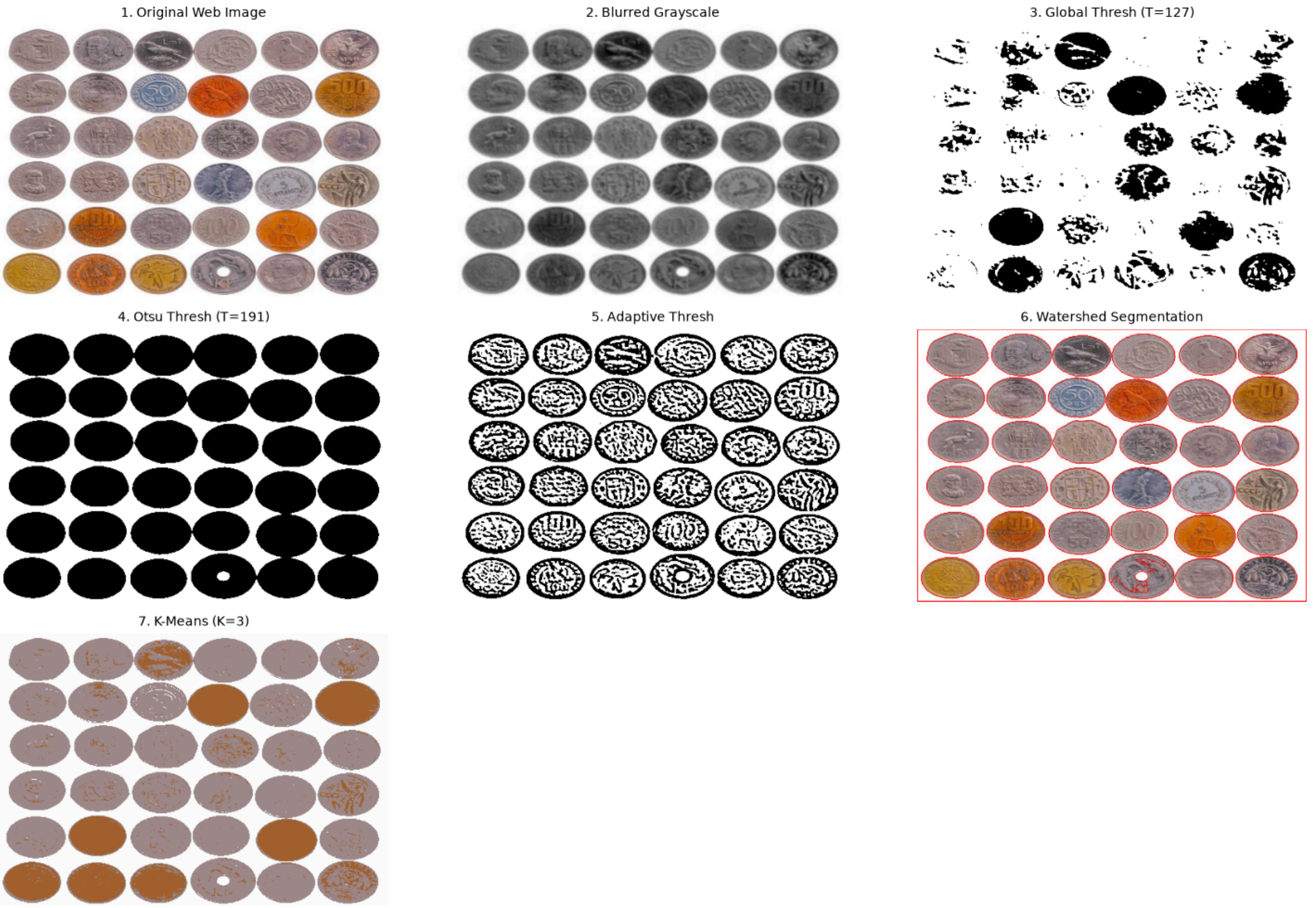
# Step 7: K-Means Clustering Segmentation (K=3)
t_start = time.time()
pixel_values = img_rgb.reshape((-1, 3))
pixel_values = np.float32(pixel_values)
k = 3
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)
_, labels, centers = cv2.kmeans(pixel_values, k, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS)
centers = np.uint8(centers)
segmented_data = centers[labels.flatten()]
kmeans_img = segmented_data.reshape((img_rgb.shape))
time_kmeans = (time.time() - t_start) * 1000

# Steps 8 & 9: Visual Output Plotting
plt.figure(figsize=(15, 10))

titles = [
    '1. Original Web Image', '2. Blurred Grayscale',
    '3. Global Thresh (T=127)', f"4. Otsu Thresh (T={int(otsu_val)})",
    '5. Adaptive Thresh', '6. Watershed Segmentation', f'7. K-Means (K={k})'
]
images = [img_rgb, blurred, global_thresh, otsu_thresh, adaptive_thresh, watershed_img, kmeans_img]
```

```
for i in range(7):
    plt.subplot(3, 3, i + 1)
    if len(images[i].shape) == 2:
        plt.imshow(images[i], cmap='gray')
    else:
        plt.imshow(images[i])
    plt.title(titles[i], fontsize=10)
    plt.axis('off')

plt.tight_layout()
plt.show()
```



Step 8: Comparative Table

Technique	Precision	Overlap/Lighting Handling	Speed
Global ($T = 127$)	Low	Fails under varied lighting/contrast	Fastest
Otsu ($T = 191$)	Medium	Good global binary split; merges touching objects	Fast
Adaptive	Medium	Handles uneven light well; creates interior noise	Fast
Watershed	High	Best; accurately separates touching objects	Moderate
K-Means ($K = 3$)	High (Color)	Robust to light shifts via color grouping	Slowest

Step 9: Key Observations

- **Global:** Unreliable due to rigid thresholding across diverse pixel intensities.
- **Otsu:** Automatically calculates optimal threshold ($T = 191$) for bimodal separation.
- **Adaptive:** Excels at local edge detail under non-uniform illumination.
- **Watershed:** Most accurate for object separation (red boundary markers).
- **K-Means:** Effectively partitions image by color attributes (copper vs. silver).

Step 10: Conclusion & Applications

- **Takeaway:** Simple thresholding offers fast binary separation, Watershed solves object overlap, and K-Means handles multi-color region grouping.
- **Applications:** Medical imaging (tumor/cell detection), autonomous driving (lane/pedestrian separation), industrial quality inspection (defect tracking), and satellite land classification.